

AI FAQ Assistant API

AI-Powered FAQ and Customer Support Automation

Project Description:

The AI FAQ Assistant API is a REST API backend for FAQ and customer-support management. It provides a centralized backend for storing and retrieving FAQ content, with secure user authentication, FAQ CRUD operations, FAQ search and AI-assisted FAQ generation.

The application is built using Node.js and Express.js for the API layer, MongoDB for data storage, Mongoose for database schemas and validation, JWT and bcrypt for authentication and password security, and Google Gemini AI for AI-powered FAQ generation. The API is tested using Thunder Client and runs on port 5000.

Scenarios

Scenario 1: A new user registers through the authentication API and logs in to receive authentication access. Protected API operations can then be accessed using the authenticated request flow.

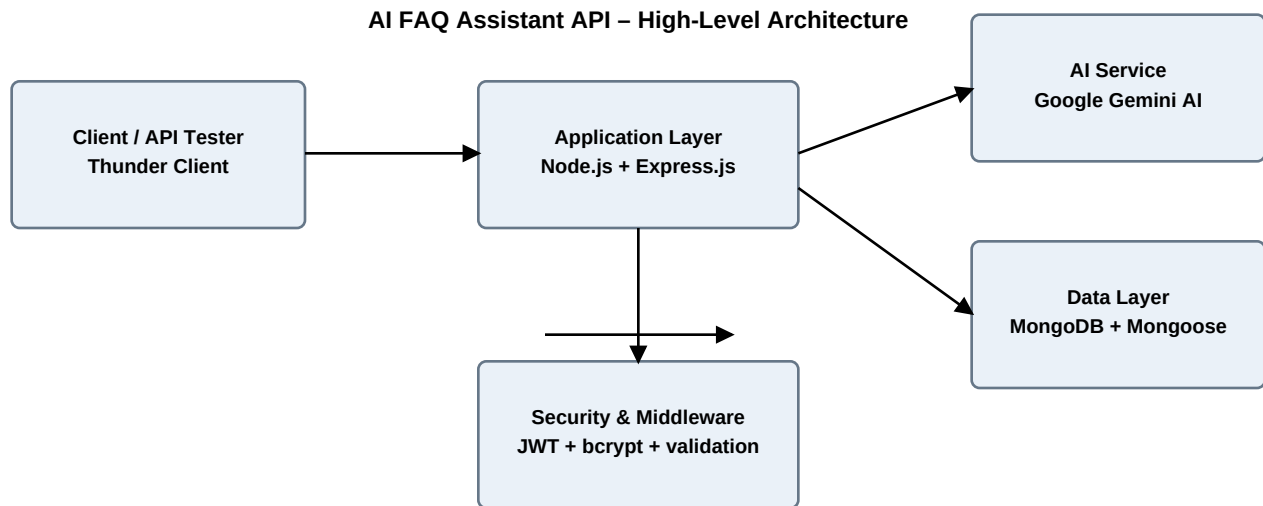
Scenario 2: An authenticated user creates, retrieves, updates, or deletes FAQ records. The centralized backend stores FAQ information in MongoDB and supports searching FAQs by query.

Scenario 3: A user sends a request to the AI generation endpoint. The backend connects to Google Gemini AI and returns AI-assisted FAQ content for customer-support use.

Project Information

Project Title	AI FAQ Assistant API
Team ID	SWTID-2026-1297
Team Size	4
Team Leader	SRIPRIYA P
Role	Team Leader & Backend Developer
Project	Naan Mudhalvan Backend Development Project

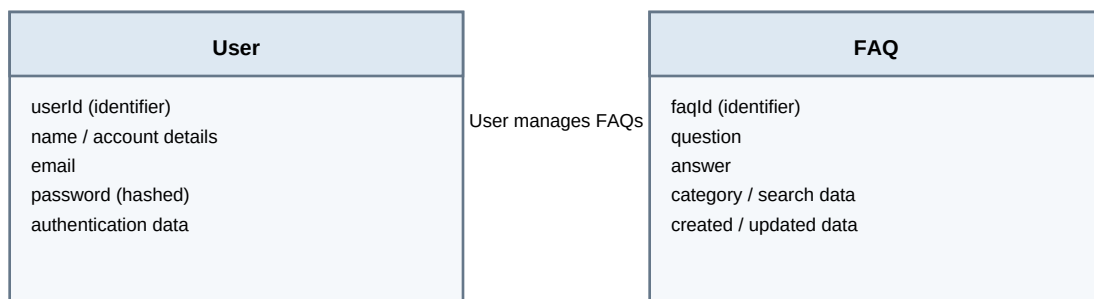
Technical Architecture:



Description: The application follows a modular backend / MVC-style structure that separates routing, business logic and data access. Express handles HTTP requests, controllers contain authentication, FAQ and AI logic, middleware handles authentication, validation and errors, models define User and FAQ schemas, routes define API endpoints, the Gemini service connects the backend with Google Gemini AI, and MongoDB stores user and FAQ information.

Database: Because the project uses MongoDB, an ER-style data relationship diagram is included below to show the main stored entities and their relationship.

ER Diagram / Data Model



The documentation identifies **User** and **FAQ** as the main Mongoose models. The relationship shown represents the application flow in which authenticated users perform FAQ operations.

Pre-requisites:

- **Node.js v18 or above:** Required for the backend runtime.
- **npm:** Used for package installation and project scripts.
- **MongoDB / MongoDB Atlas:** Required for database storage.
- **Visual Studio Code:** Development environment.
- **Thunder Client:** Used for API testing.
- **REST API and Express.js knowledge:** Useful for routes, requests and JSON responses.
- **JWT and bcrypt basics:** Required to understand authentication and password hashing.
- **Google Gemini AI familiarity:** Required for the AI FAQ generation integration.

Project Workflow:

Milestone 1: Project Setup and Architecture

- **Activity 1.1:** Set up Node.js, npm, MongoDB / MongoDB Atlas and the development environment.
- **Activity 1.2:** Define the modular backend structure with routes, controllers, middleware, models, services and utilities.
- **Activity 1.3:** Configure environment variables for database and Gemini credentials.
- **Activity 1.4:** Establish the Express server and port 5000.

Milestone 2: Authentication Development

- **Activity 2.1:** Implement user registration and login.
- **Activity 2.2:** Add JWT authentication and protected routes.
- **Activity 2.3:** Use bcrypt for password hashing.

Milestone 3: FAQ Management

- **Activity 3.1:** Implement create, read, update and delete FAQ operations.
- **Activity 3.2:** Implement FAQ search and categorization.
- **Activity 3.3:** Add validation and centralized error handling.

Milestone 4: AI Integration

- **Activity 4.1:** Connect the backend to Google Gemini AI through geminiService.js.
- **Activity 4.2:** Implement the AI FAQ generation endpoint.

Milestone 5: API Testing

- **Activity 5.1:** Test authentication, FAQ operations, search, validation and AI generation using Thunder Client.
- **Activity 5.2:** Verify JSON responses and protected API access.

Milestone 6: Conclusion and Future Enhancement

Milestone 1 & 2: Setup and Authentication

Activity 1.1: Development Environment Setup

The project requires Node.js v18 or above, npm, MongoDB or MongoDB Atlas, Visual Studio Code and Thunder Client. The backend dependencies are installed with npm and the application is started with the configured project script.

Installation command:

```
npm install
```

The required database and Gemini credentials are configured through the **.env** file. The application is started using:

```
npm start
```

The current application runs on **port 5000**.

Activity 2.1: User Registration

The registration endpoint creates a new user account. The API accepts the required registration data, validates the request and stores the user information using the User model.

Endpoint:

```
POST /api/auth/register
```

Activity 2.2: User Login and JWT Authentication

The login endpoint authenticates a registered user and provides authentication access. JWT is used for authentication and protected routes are handled through authentication middleware.

Endpoint:

```
POST /api/auth/login
```

Activity 2.3: Password Security

The project uses **bcrypt** for password hashing. This keeps passwords from being stored as plain text in the database.

Authentication flow

- User sends registration or login request.
- Authentication controller processes the request.
- Password security is handled using bcrypt.
- Successful login provides authentication access through JWT.
- Protected routes use authentication middleware before allowing access.

Milestone 3: FAQ Management

Activity 3.1: FAQ Create, Read, Update and Delete

The FAQ module provides CRUD operations for centralized FAQ storage and retrieval. FAQ data is represented using the Mongoose FAQ model and is stored in MongoDB.

Core FAQ endpoint:

```
POST /api/faqs → Create a FAQ
```

The project documentation also specifies FAQ retrieval, update and delete operations as part of the FAQ CRUD functionality.

Activity 3.2: FAQ Search and Categorization

The backend supports searching FAQ records through a query parameter. The documented example is:

```
GET /api/faqs/search?q=configure
```

This allows the client to request FAQ information matching the supplied search query. The project overview also identifies FAQ categorization as a supported feature.

Activity 3.3: Validation and Error Handling

Validation middleware is included in the project structure and centralized error handling is implemented through `errorMiddleware.js`. These components help keep request validation and error responses consistent across the backend.

Relevant project modules

- `models/FAQ.js` – FAQ schema and data model.
- `controllers/faqController.js` – FAQ business logic.
- `routes/faqRoutes.js` – FAQ API routes.
- `middleware/validationMiddleware.js` – request validation.
- `middleware/errorMiddleware.js` – centralized error handling.
- `utils/helpers.js` – utility functions.

Milestone 4: AI Integration

Activity 4.1: Gemini Service

Google Gemini AI is used for AI-powered FAQ generation. The project separates this integration into a dedicated service file, `src/services/geminiService.js`, keeping AI communication separate from the controllers and routes.

Activity 4.2: AI FAQ Generation Endpoint

The documented AI endpoint is:

```
POST /api/ai/generate-faq
```

The endpoint is intended to generate FAQ content using Gemini AI. The AI controller handles the corresponding business logic and the Gemini service provides the connection to the external AI service.

AI request flow

- Thunder Client sends a request to the AI generation endpoint.
- The Express route forwards the request to the AI controller.
- The AI controller invokes the Gemini service.
- Google Gemini AI processes the request.
- The generated FAQ content is returned through the REST API response.

Relevant project modules

- `routes/aiRoutes.js` – AI endpoint routing.
- `controllers/aiController.js` – AI business logic.
- `services/geminiService.js` – Google Gemini AI integration.

Environment configuration

The project documentation states that database and Gemini credentials must be configured in the `.env` file. The `.env` file should not be exposed in a public repository.

Milestone 5: Backend Structure and API Layer

The backend uses a modular structure that separates routing, controllers, middleware, models, services and utilities. This makes the API easier to maintain and extend.

Folder Structure

```
Code Files
├── src
│   ├── config
│   ├── db.js
│   ├── controllers
│   │   ├── aiController.js
│   │   ├── authController.js
│   │   ├── faqController.js
│   ├── middleware
│   │   ├── authMiddleware.js
│   │   ├── errorMiddleware.js
│   │   ├── validationMiddleware.js
│   ├── models
│   │   ├── FAQ.js
│   │   ├── User.js
│   ├── routes
│   │   ├── aiRoutes.js
│   │   ├── authRoutes.js
│   │   ├── faqRoutes.js
│   ├── services
│   │   ├── geminiService.js
│   ├── utils
│   │   ├── helpers.js
│   ├── app.js
│   ├── server.js
│   ├── .env
│   ├── package.json
│   ├── README.md
│   └── test_api.js
```

Layer responsibilities

Layer / Component	Responsibility
Routes	Defines authentication, FAQ and AI endpoints.
Controllers	Contains authentication, FAQ and AI business logic.
Middleware	Handles authentication, validation and errors.
Models	Defines User and FAQ schemas using Mongoose.
Services	Connects the backend with Google Gemini AI.
Config	Contains database connection configuration.
Server / App	Initializes the Express application and server.

API Documentation

Authentication APIs

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Login and receive authentication access

FAQ APIs

Method	Endpoint	Purpose
POST	/api/faqs	Create a FAQ
GET	/api/faqs/search?q=configure	Search FAQs
CRUD	FAQ routes	Create, read, update and delete FAQ records

AI API

Method	Endpoint	Purpose
POST	/api/ai/generate-faq	Generate FAQ content using Gemini AI

API response format

The project is a REST API backend and returns JSON responses. The exact response payloads are not fully specified in the supplied documentation, so this document does not invent additional request or response fields.

Testing tool

API testing is performed using **Thunder Client**, including registration and login, FAQ creation and retrieval, FAQ search, AI FAQ generation, authentication and validation.

API Testing and Verification

The supplied project documentation states that manual API testing is used to verify the main backend functions.

Test Areas

- **User registration:** Verify that a new user can be registered through the authentication endpoint.
- **User login:** Verify that a registered user can log in and receive authentication access.
- **Protected API access:** Verify that protected routes use authentication.
- **FAQ CRUD:** Verify creation, retrieval, update and deletion of FAQ records.
- **Search:** Verify FAQ search using the documented query endpoint.
- **AI FAQ generation:** Verify the Gemini-powered generation endpoint.
- **Validation:** Verify request validation and error handling.

Thunder Client Testing Flow

1. Start the backend with **npm start**.
2. Confirm that the server is listening on port **5000**.
3. Open Thunder Client in Visual Studio Code.
4. Test the registration and login endpoints.
5. Test protected FAQ endpoints using the authentication flow.
6. Test FAQ search and AI FAQ generation.
7. Check the HTTP status and JSON response for each request.

Expected verification

Successful testing should demonstrate that the documented API routes respond correctly, authentication is applied to protected operations, FAQ information can be managed, search works, and the AI generation endpoint can communicate with the configured Gemini service.

Note: The source documentation contains headings for screenshots of terminal, registration, login, user profile and Gemini output, but the supplied DOCX does not provide readable screenshot image content. Therefore, this generated PDF does not invent or recreate those screenshots.

Running the Application

Activity 5.1: Install Dependencies

Open the project folder in Visual Studio Code and install the dependencies using:

```
npm install
```

Activity 5.2: Configure Environment Variables

Configure the required database and Gemini credentials in the `.env` file. Keep credentials private and do not expose them in the public source repository.

Activity 5.3: Start the Backend Server

Run:

```
npm start
```

After successful startup, MongoDB connects and the server listens on port 5000.

Activity 5.4: Test Through Thunder Client

Use Thunder Client to send HTTP requests to the API endpoints. Because this project focuses on backend API development, there is no separate graphical user interface; Thunder Client is used to interact with the API.

Application Execution Flow

- Node.js starts the Express server.
- The database configuration establishes the MongoDB connection.
- Express registers the authentication, FAQ and AI routes.
- Middleware validates requests and protects authenticated operations.
- Controllers process business logic.
- Mongoose models read and write data in MongoDB.
- The Gemini service handles AI-powered FAQ generation.
- JSON responses are returned to Thunder Client.

Technologies Used

Technology	Purpose
Node.js	Backend runtime
Express.js	REST API framework
MongoDB	Database
Mongoose	ODM and schema validation
JWT	Authentication
bcrypt	Password hashing
Google Gemini AI	AI-powered FAQ generation
Thunder Client	API testing
Visual Studio Code	Development environment

Why these technologies are used

Node.js + Express.js: Provide the server runtime and REST API framework.

MongoDB + Mongoose: Provide document storage and schema-based access to User and FAQ data.

JWT + bcrypt: Provide authentication and password protection.

Google Gemini AI: Provides the AI-powered FAQ generation capability.

Thunder Client: Provides direct API request testing during development.

The project documentation describes the system as a backend API and does not include a separate frontend interface.

Known Issues and Future Enhancements

Known Issues

- No separate frontend interface is included.
- The application requires correct environment-variable configuration.
- AI generation depends on valid Gemini API configuration and availability.

Future Enhancements

- Add a web-based frontend.
- Improve semantic search and result ranking.
- Add pagination and analytics.
- Expand AI-based customer-support features.
- Deploy the API to a cloud platform.

Possible Enhancement Direction

The documented future work focuses on improving the user-facing experience, search quality, analytics, AI customer-support capabilities and deployment. These enhancements can build on the current modular separation of routes, controllers, middleware, models and services.

Project Demonstration

Demo video link:

https://drive.google.com/file/d/17ovLwNVU9F897t-4__6F994D8HChFmXS/view?usp=drivesdk

Team Information

Name	Email ID	Role
SRIPRIYA P	24bsccs19@cttewc.edc.in	Team Leader & Backend Developer
Harini Karthiga G	24bsccs09@cttewc.edu.in	Documentation
Monisha T	24bsccs35@cttewc.edu.in	Documentation
Daisy rani V	24bsccs34@cttewc.edu.in	PPT

Project Scope

The project scope is focused on backend development for FAQ and customer-support automation. It combines REST API development, database management, authentication, search and AI-assisted FAQ generation in one modular application.

Project Outcome

The documented implementation provides the core backend components required for user management, FAQ management, search and AI FAQ generation, with manual API testing through Thunder Client.

Conclusion

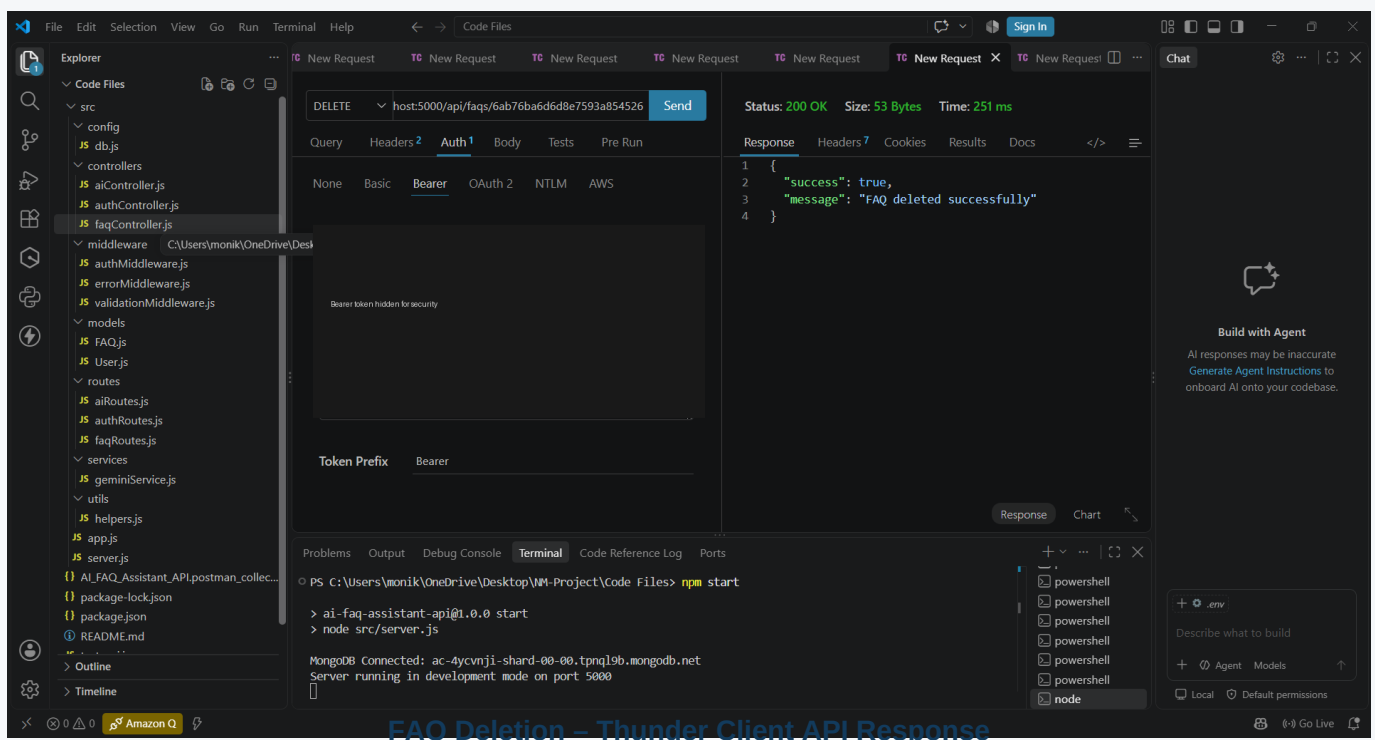
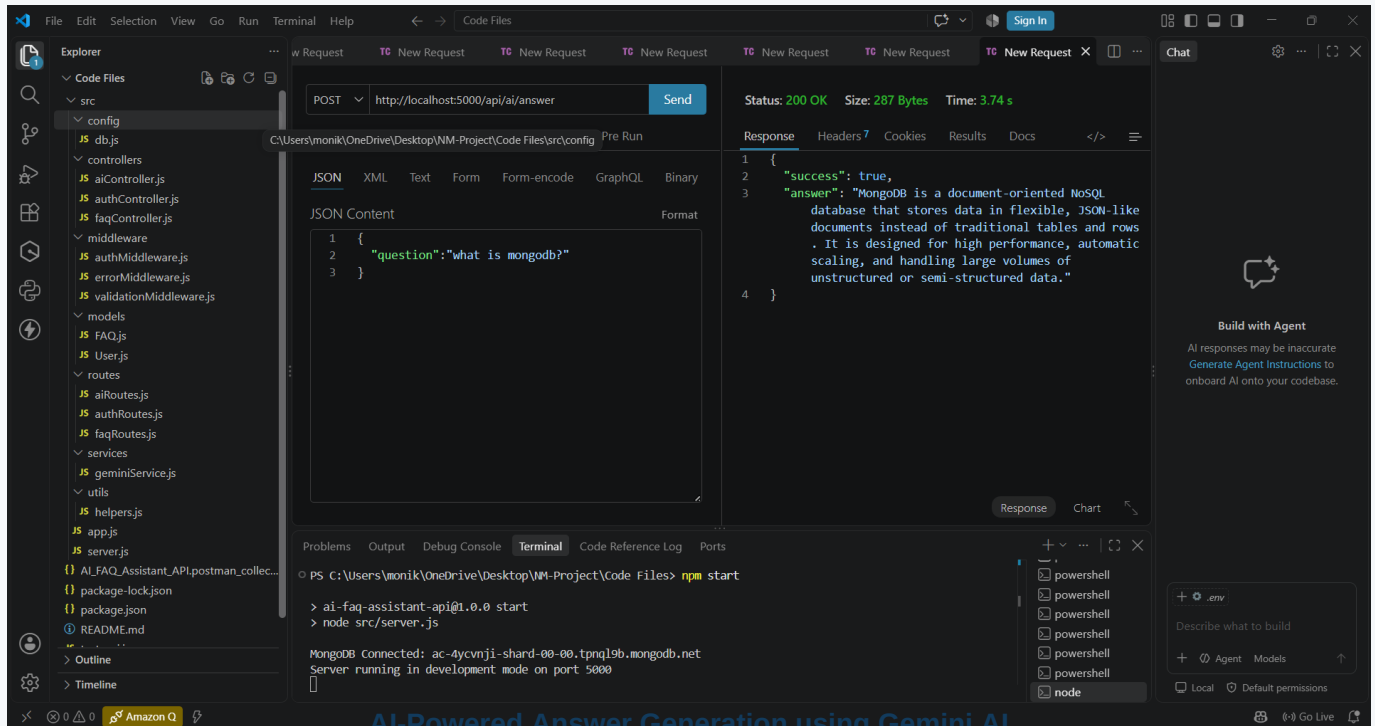
The AI FAQ Assistant API is a backend solution designed to centralize FAQ and customer-support management. It uses Node.js and Express.js to provide REST APIs, MongoDB and Mongoose for data storage, JWT and bcrypt for authentication and password security, and Google Gemini AI for AI-powered FAQ generation.

The modular project structure separates routes, controllers, middleware, models, services and utilities, making the application easier to maintain and extend. The main documented capabilities include user registration and login, protected routes, FAQ CRUD operations, FAQ search and categorization, validation, centralized error handling and AI-assisted FAQ generation.

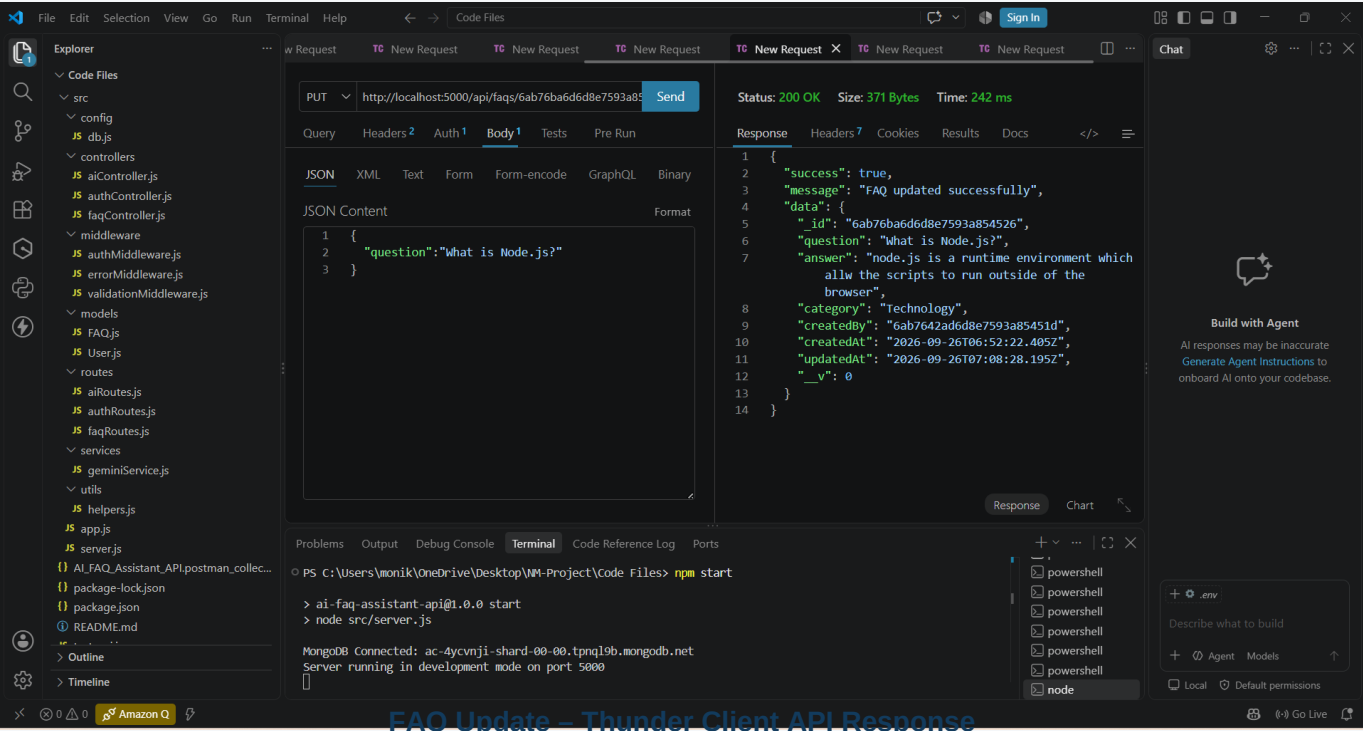
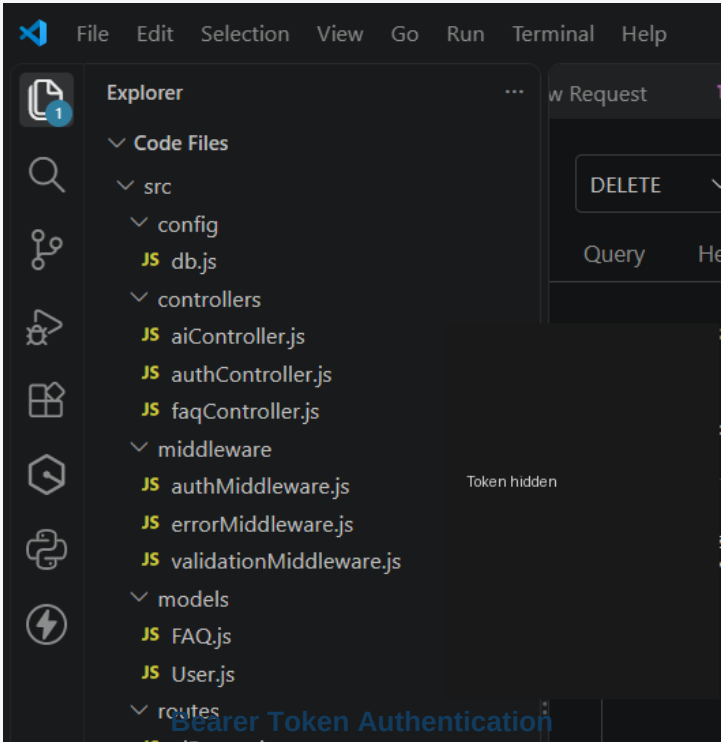
The project is tested through Thunder Client, with the backend running on port 5000. The supplied documentation identifies a web frontend, improved semantic search, pagination and analytics, expanded AI customer-support features, and cloud deployment as future enhancements.

End of Project Documentation

Screenshots / Demo

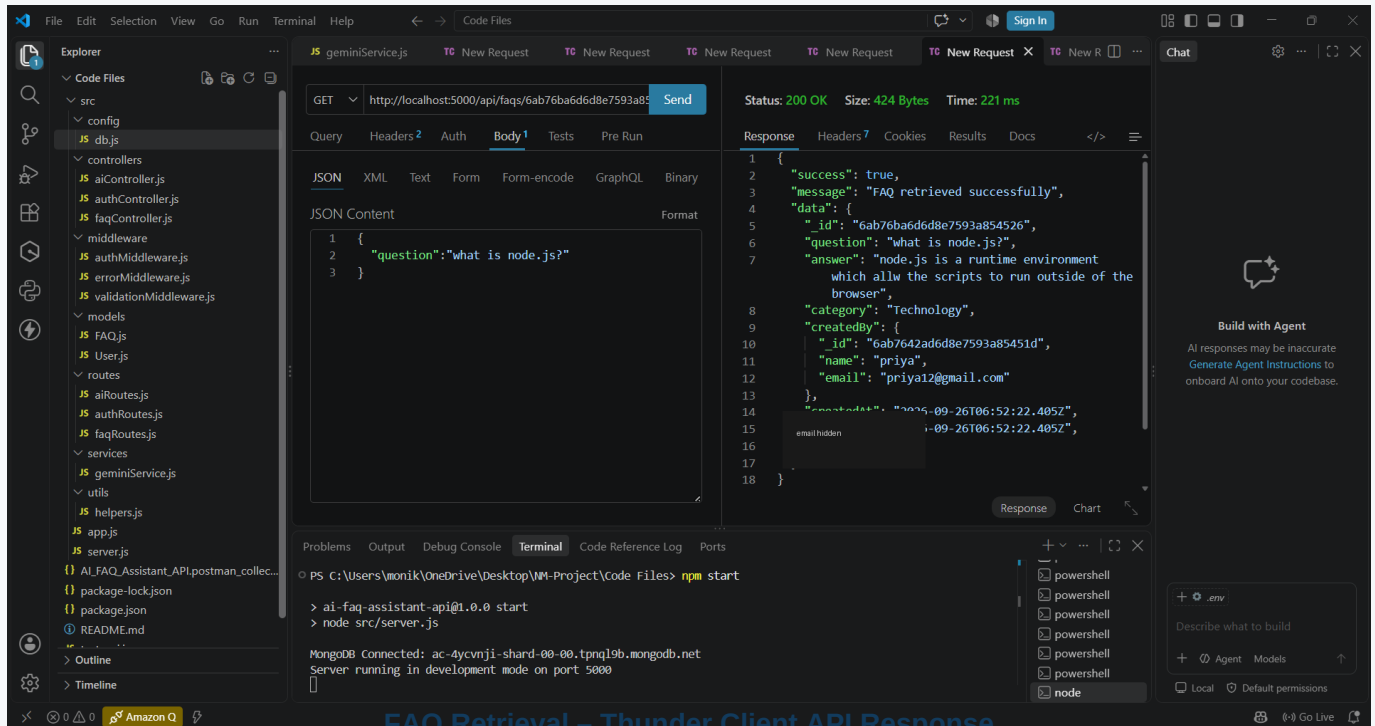


Screenshots / Demo

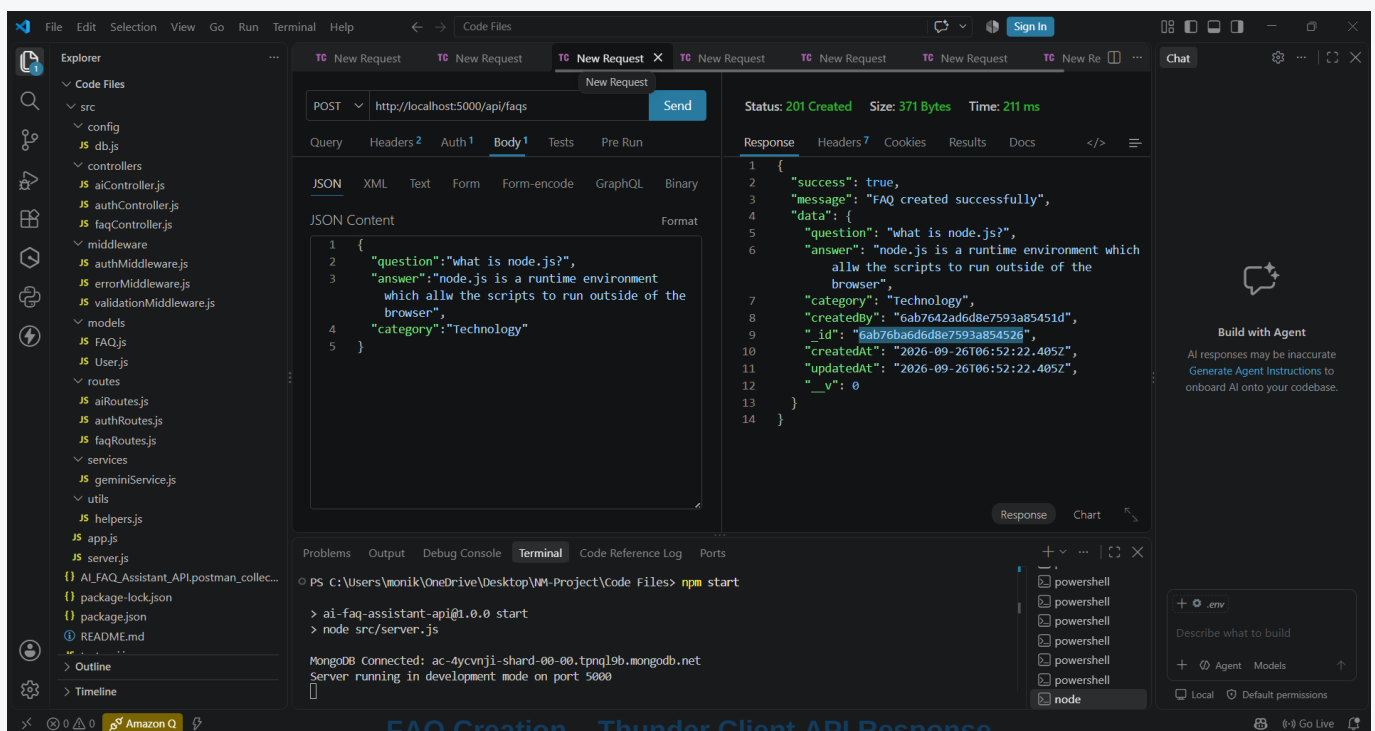


FAQ Update – Thunder Client API Response

Screenshots / Demo



FAQ Retrieval – Thunder Client API Response



FAQ Creation – Thunder Client API Response

The screenshot displays the VS Code interface with a REST client request to `http://localhost:5000/api/auth/profile`. The request is a GET method with a Bearer token. The response is a JSON object containing user profile information. The terminal shows the command `npm start` and the server output.

Request:

```
GET http://localhost:5000/api/auth/profile
```

Headers:

```
Auth 1
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhYjc2NDIhZDZkOGU3NTkzYTg1NDUxZCIsImhhdCI6MTc5MDQwMzYyNiwiZXhwIjojNzkyOTk1NjIzQzQ7u49S3m36mFBK4uHnSsTOhgXW3m1c4QkiFlyYQlp6s
```

Response:

```
200 OK
Size: 230 Bytes
Time: 97 ms

{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "id": "6ab7642ad6d8e7593a85451d",
    "name": "priya",
    "email": "priya12@gmail.com",
    "createdAt": "2026-09-26T06:20:26.818Z",
    "updatedAt": "2026-09-26T06:20:26.818Z",
    "__v": 0
  }
}
```

Terminal:

```
PS C:\Users\monik\OneDrive\Desktop\WM-Project\Code Files> npm start

> ai-faq-assistant-api@1.0.0 start
> node src/server.js

MongoDB Connected: ac-4ycvni-shard-00-00.tpnql9b.mongodb.net
Server running in development mode on port 5000
```